The `collection.where` method finds objects within the collection based on the conditions supplied but the objects are loaded lazily meaning that the database is queried only when the object(s) are accessed.

```
@open_orders = @customer.orders.where(open: true) # No query yet
@open_order = @open_orders.first # Now the database will be queried
```

4.3.1.13 `collection.exists?(...)`

The `collection.exists?` method checks whether an object meeting the supplied conditions exists in the collection. It uses the same syntax and options as `ActiveRecord::Base.exists?`.

4.3.1.14 `collection.build(attributes = {}, ...)`

The `collection.build` method returns one or more new objects of the associated type. These objects will be instantiated from the passed attributes, and the link through their foreign key will be created, but the associated objects will *not* yet be saved.

```
@order = @customer.orders.build(order_date: Time.now,
                                order_number: "A12345")
```

4.3.1.15 `collection.create(attributes = {})`

The `collection.create` method returns a new object of the associated type. This object will be instantiated from the passed attributes, the link through its foreign key will be created, and, once it passes all of the validations specified on the associated model, the associated object *will* be saved.

```
@order = @customer.orders.create(order_date: Time.now,
                                 order_number: "A12345")
```

4.3.1.16 `collection.create!(attributes = {})`

Does the same as `collection.create` above, but raises `ActiveRecord::RecordInvalid` if the record is invalid.

4.3.2 Options for `has_many`

While Rails uses intelligent defaults that will work well in most situations, there may be times when you want to customize the behavior of the `has_many` association reference. Such customizations can easily be accomplished by passing options when you create the association. For example, this association uses two such options:

```
class Customer < ActiveRecord::Base
  has_many :orders, dependent: :delete_all, validate: :false
end
```

The `has_many` association supports these options:

- `:as`
- `:autosave`
- `:class_name`
- `:counter_cache`
- `:dependent`
- `:foreign_key`
- `:inverse_of`
- `:primary_key`
- `:source`
- `:source_type`
- `:through`
- `:validate`

4.3.2.1 `:as`

Setting the `:as` option indicates that this is a polymorphic association, as discussed earlier in this guide.

4.3.2.2 `:autosave`

If you set the `:autosave` option to `true`, Rails will save any loaded members and destroy members that are marked for destruction whenever you save the parent object.

4.3.2.3 `:class_name`